

Migration as a Steady-State Execution Primitive: A Runtime for Virtual Stationarity in Low-Earth Orbit

Daniel Kleebinder[✉]

d.kleebinder@dsg.tuwien.ac.at

Distributed Systems Group, TU Wien
Vienna, Austria

Alireza Furutanpey

a.furutanpey@dsg.tuwien.ac.at

Distributed Systems Group, TU Wien
Vienna, Austria

Stefan Nastic

s.nastic@dsg.tuwien.ac.at

Distributed Systems Group, TU Wien
Vienna, Austria

Abstract

Low Earth Orbit (LEO) satellite mega-constellations enable orbital edge computing for latency-critical geocentric applications, but satellite motion compresses line-of-sight communication windows to mere minutes. To maintain logical geographic proximity – a paradigm termed Virtual Stationarity – applications must continuously migrate across passing satellites in the constellation. Traditional live migration mechanisms optimize individually initiated relocations or replication, but not the continuous handover between successive hosts that orbital motion requires. We reformulate live migration as a steady-state execution model and introduce a novel virtual stationarity runtime paradigm. We formalize the viability of continuous migration through a lightweight model capturing latency and availability SLOs, continuous migration overhead, and the convergence bound of state replay. To satisfy these constraints, we derive a Virtual Stationarity Runtime (VSR) framework whose Migration Membrane combines Capability Virtualization and Continuous State Projection to rebind host resources and reconstruct evolving execution on a successor. Evaluation on physical heterogeneous hosts shows p_{95} downtime below 6 ms across 300 recurrent handovers, migration time effectively invariant to memory-write rate, and no cumulative growth in migration time or continuation state; a 1,600-satellite simulation further confirms completion within orbital handover windows.

CCS Concepts

• **Computer systems organization** → **Heterogeneous (hybrid) systems; Distributed architectures; Reliability**; • **Networks** → **Network dynamics**.

Keywords

edge computing, satellite networks, live migration

ACM Reference Format:

Daniel Kleebinder, Alireza Furutanpey, and Stefan Nastic. 2026. Migration as a Steady-State Execution Primitive: A Runtime for Virtual Stationarity in Low-Earth Orbit. In *Proceedings of The 16th International Conference on the Internet of Things (IoT 2026) (IoT 2026)*. ACM, New York, NY, USA, 9 pages. <https://doi.org/XXXXXXX.XXXXXXX>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

IoT 2026, Newcastle upon Tyne, UK

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/2018/06
<https://doi.org/XXXXXXX.XXXXXXX>

1 Introduction

By mid-2026, more than 16,200 active payloads were in orbit, more than twice the number recorded three years earlier [23]. This rapid proliferation is catalyzing a fundamental shift in spaceborne computing [12]: recent research increasingly envisions satellites not only as passive communication relays but as active compute nodes interconnected via optical Inter-Satellite Links (ISLs) and linked to the ground through Ground-to-Satellite Links (GSLs), forming a unified distributed computing continuum that bridges space and Earth [3, 12]. At altitudes between 80 km and 2,000 km [27], LEO satellites yield single-digit-millisecond propagation delays to ground users [3]. Hosting latency-critical workloads onboard passing spacecraft can therefore exploit this physical proximity to satisfy stringent real-time SLOs [24, 29, 31].

However, extreme satellite velocities ($\approx 27,000$ km/h) compress direct line-of-sight contact windows over target ground regions to mere minutes [4, 42]. To maintain logical proximity, applications must therefore continuously migrate across passing spacecraft at high frequencies, elevating live migration from an exceptional terrestrial operation [1, 16] to a steady-state LEO execution primitive. As §2 demonstrates, canonical migration strategies fail during high-frequency handovers by causing prolonged downtime, non-convergence under write pressure, or lingering predecessor dependencies. Advanced terrestrial mechanisms rely on assumptions – such as kernel homogeneity, deterministic behavior, or extended predecessor reachability [2, 26, 32] – that collapse in orbit, where nodes differ in Instruction Set Architecture (ISA) and payload capabilities, migration budgets are tightly bounded, and source connectivity terminates abruptly.

To align orbital transience with terrestrial stability, prior work introduced Virtual Stationarity (VS) to logically anchor applications over fixed ground targets [3]. We extend VS from a placement objective into a runtime model that establishes live migration as a steady-state execution primitive. Specifically, we make three core contributions: (i) we formalize the system model and feasibility conditions governing recurrent orbital migration, establishing bounds of latency and availability SLOs, hosting windows, and cumulative steady-state overhead (§4.1); (ii) we design a VSR framework that separates external orchestration from runtime handover and defines a four-phase Prepare–Reconstruct–Project–Commit protocol for recurrent orbital migration (§4.2); and (iii) we introduce the *Migration Membrane*, comprising two enabling mechanisms: Capability Virtualization ($\mathcal{V}$) to dynamically rebind host-dependent resources across heterogeneous satellite configurations, and Continuous State Projection ($\mathcal{P}$) to asynchronously prepare successor nodes while the source continues executing (§4.3). We instantiate the framework and membrane within WAMR, implementing an

authority fence for single-writer cutovers across heterogeneous ISAs (§4.4).

We evaluate VSR on a physical Raspberry Pi 5 testbed, through bidirectional x86-64/ARM64 migration trials, and under SGP4-derived orbital schedules (§5). Across 300 recurrent handovers, p_{95} downtime remains below 6 ms without cumulative performance degradation, while all cross-ISA trials resume successfully. Under realistic orbital profiles, migrations complete in 2.8–4.5 s between handovers spaced 83–242 s apart, yielding an availability of $A_d \geq 99.997\%$.

2 Motivation

Although GSLs between ground stations and satellites can achieve near-optimal propagation latencies of under 10 ms [35], contemporary satellite networks operate primarily as terrestrial backhubs, introducing substantial routing overheads. This section analyzes the operational challenges of achieving VS through an Industry 5.0 offshore wind farm maintenance use case.

2.1 XR-Assisted Offshore Maintenance

Consider an offshore wind farm operating beyond terrestrial edge connectivity. Technicians conducting turbine maintenance employ head-mounted Extended Reality (XR) devices streaming real-time video, telemetry, and equipment state to an edge application providing interactive visual overlays and remote expert guidance [7, 33]. Sustaining these interactive workflows demands strict, ultra-low-latency execution; service degradation or latency jitter directly impairs task efficiency and operational safety. Hosting this application onboard passing LEO satellites provides low-latency compute access without requiring capital-intensive offshore edge hardware. However, a typical maintenance window far exceeds the few-minute contact window of an individual satellite. Unassisted static deployment – pinning the application to its initial host satellite – causes the network path to stretch as the spacecraft moves past the horizon, necessitating a multi-hop inter-satellite relay which quickly causes latency Service-Level Objective (SLO) violations.

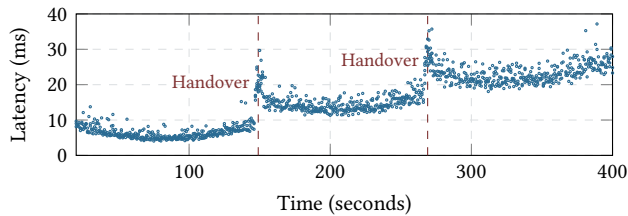


Figure 1: End-to-end latency during satellite pass-over. Loss of direct line-of-sight and relay rerouting at 150 s and 270 s cause step-wise increases in baseline latency and jitter.

We empirically demonstrate this degradation in Figure 1. During the initial direct line-of-sight pass-over, Round-Trip-Time (RTT) ranges between 4 ms and 11 ms (p_5 and p_{95}) due to dynamic slant distances and emulated network jitter. As orbital mechanics drive the host satellite below the horizon, traffic is rerouted over inter-satellite relays. This increases median RTT from 6.6 ms ($\sigma = 2.4$ ms) to 14.4 ms ($\sigma = 2.9$ ms) for a single relay hop, reaching 23.1 ms ($\sigma = 3.3$ ms) across two hops within five minutes. Here, network

RTT alone exceeds 20 ms, exhausting the Motion-to-Photon (M2P) budget before sensing, processing, and rendering delays are included. This can induce a visual-vestibular misalignment, which increases the risk of simulator sickness that severely restricts technician operational time and safety [5, 17, 28, 36]. Without migration, the worst-case co-planar route (180° from the target) even requires 16 ISL hops across a 550 km altitude shell. Extrapolating our measured p_{95} penalty yields a propagation-only RTT of 152.4 ms – a delay that renders interactive XR entirely unviable.

From the technician’s perspective, the application should remain anchored directly above the wind farm throughout the maintenance session. Its logical location and active state must remain invariant despite the rapid physical movement of underlying spacecraft.

2.2 Migrating the Application

Continuous, high-frequency migration converts live migration from an exception-handling mechanism into a core steady-state execution primitive. To evaluate migration under this assumption, we extended WAMR to support three canonical migration paradigms. We omit kernel tools such as CRIU as they require OS configurations that are not supported on constrained edge hardware [11]. We deployed the experiment on a two-node Raspberry Pi 5 testbed, emulating satellite nodes receiving UDP telemetry from a VR *Gaming Dataset* trace ($\approx 4,000$ pkts/s; ≈ 5 MB/s) [25]. Figure 2 illustrates the performance trade-offs of these mechanisms during handover.

Checkpoint/Restore halts execution on the source node, serializes the complete memory and CPU register state into a snapshot, transfers it across the network, and restores execution on the target host [6, 8]. Despite its conceptual simplicity, Figure 2a shows that completely pausing application execution during state transfer induces a full service blackout with 100% packet loss. *Pre-Copy* transfers an initial memory snapshot to the target before repeatedly transmitting dirtied memory pages while the application continues to execute on the source [15, 37]. While this strategy maintains low baseline packet loss (Figure 2b), high memory write rates can prevent iterative convergence [9]. Consequently, the overall migration duration inflates past 10 s, consuming a prohibitive fraction of the 15 s beam-switch window typical for Starlink [42]. *Post-Copy* pauses the source instance only long enough to transfer minimal execution registers, immediately spawning the process on the target host without its memory. As execution resumes, missing memory pages are fetched reactively from the source host [18, 30]. Although initial downtime remains negligible, Figure 2c demonstrates that fetching unmapped memory on demand across the inter-satellite link triggers severe latency jitter and sustained packet loss of $\approx 75\%$.

Given the brief contact windows and the compute and energy limits of satellites, traditional terrestrial trade-offs that sacrifice migration time for downtime, or vice versa, are prohibitive.

3 Related Work

Stateful migrations are commonly implemented as individually triggered infrastructure operations [1, 19, 34]. ReSync [32] couples CRIU checkpoint/restore with buffered application-input replay and coordinates migration with mobile-network handover. Instead, VSR leaves placement and timing to an external orchestrator, and projects observed host-call returns and application-memory effects.

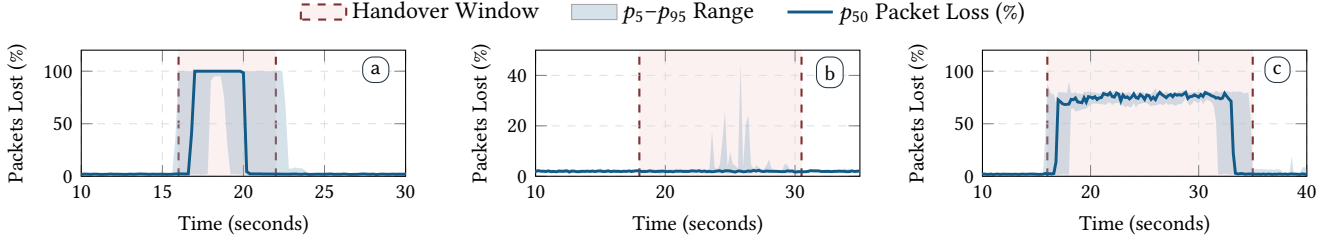


Figure 2: Packet loss during inter-satellite handover of a 256 MB UDP-based XR workload. Red shading marks handover windows; lines and bands show the median and p5-p95 range. (a) Checkpoint/restore causes a complete blackout. (b) Pre-copy limits loss but prolongs handover. (c) Post-copy incurs sustained loss from reactive page faults.

The successor can therefore reconstruct execution without repeating external effects. Voyager [26] combines CRIU memory transfer with a networked union filesystem, from which the target retrieves persistent state during lazy transfer. VSR does not federate host resources or require predecessor access after commit. It rebinds logical capabilities to target-local resources. dOS/RecShim [2] enforces deterministic process groups and records external nondeterminism for replay and replication. VSR applies the same separation between deterministic execution and external effects to project a live computation across successive authoritative hosts rather than provide general deterministic execution. ReSync, Voyager, and dOS/RecShim rely on OS- or kernel-level mechanisms and generally reproduce the same host environment at the destination. VSR transfers portable runtime state and rebinds declared capabilities, avoiding common-kernel and common-ISA requirements across satellite compute platforms with heterogeneous processors and payload capabilities.

ReVirt [13] logs nondeterministic events beneath a guest OS for instruction-level VM replay and post-intrusion analysis. Liu et al. [21] combine a copy-on-write VM checkpoint with iterative transfer and target-side full-system replay. Their design is mechanically close to continuous state projection, but runs at the VM boundary and targets a single handover. Yu et al. [41] adapt iterative logging and replay to changes above a shared Docker image. VSR instead defines a runtime protocol in which boundary events record host-call results and explicit application-memory mutations, replay suppresses new external effects, capabilities are rebound at the successor, and an authority fence prevents duplicate effects.

Bhattacharjee et al. [3] introduced VS and evaluated placement policies that keep a service near a fixed ground location. Wu et al. [39] jointly optimize task offloading and service migration, while Databelt [22] constructs a data path for orbital serverless workflows. Komet [29] repeatedly instantiates stateless functions and proactively replicates application-managed state. These systems select or prepare the next placement but do not specify how to transfer a running execution, re-establish its capabilities, and preserve a single authority across recurrent handovers. Orbital handover windows are bounded and may terminate access to the predecessor. VSR complements this placement work by accepting the successor and constraints from an external orchestrator, terminating predecessor dependence at commit, and exposing downtime, migration duration, and long-run migration occupancy for feasibility analysis.

4 A Runtime for Virtual Stationarity

We first formalize the correctness and feasibility contract for recurrent migration (§4.1). We then present the Virtual Stationarity Runtime (VSR), separating its runtime responsibilities from the external orchestration and execution-engine services it assumes (§4.2). Finally, we detail the membrane’s two enabling mechanisms (§4.3) and report one concrete realization (§4.4).

4.1 System and Feasibility Model

We model an application anchored to a ground target g over an operational interval $\mathcal{T}$, with duration $|\mathcal{T}|$. The application may execute on any satellite S_i in the constellation. Its end-to-end RTT to S_i at time t is described by the latency process $\zeta_i : \mathcal{T} \rightarrow \mathbb{R}^+ \cup \{\infty\}$, where $\zeta_i(t) = \infty$ denotes that no GSL/ISL path exists. Orbital motion makes the geometric component of ζ_i predictable, while atmospheric, processing, and queueing delays remain time-varying.

4.1.1 Continuity Contract. Let $\sigma(t) = \langle E(t), M(t), C(t), g \rangle$ denote the logical application state: execution context E , mutable state M , logical capability contract C , and geographic anchor g . Host S_i realizes this logical state as $\rho_i(t) = \pi_i(\sigma(t))$ by translating execution and binding logical capabilities to native implementations. Two realizations are equivalent if, for the same subsequent inputs, they produce the same application-visible state and external effects.

A correct handover preserves four invariants. *State continuity* requires the target to continue from an equivalent application-visible state. *Capability continuity* preserves the identity and semantics of functions classified as required when the successor is selected; the availability of optional functions may differ across hosts and is explicitly observable by the application. *Effect continuity* prevents externally visible operations from being omitted or repeated during reconstruction. Finally, *authority exclusivity* permits only one realization to produce new external effects. A successor may consequently be reconstructed while the source runs, but it remains non-authoritative until handover completes.

4.1.2 Service Objectives and Hosting Windows. The application specifies a latency SLO S_{max} and an availability SLO A_{min} . Let $D \subseteq \mathcal{T}$ contain all times at which the application does not answer within S_{max} , including execution pauses, path-update delays, and ordinary latency violations. End-to-end availability is therefore

$$A = 1 - \frac{|D|}{|\mathcal{T}|} \geq A_{min}. \quad (1)$$

A satellite is an admissible host only while its latency remains within the SLO. Starting at time t , its remaining hosting window is

$$T_{host}(S_i, t) = \sup\{\Delta t \geq 0 \mid \forall \tau \in [t, t + \Delta t] : \zeta_i(\tau) \leq S_{max}\}. \quad (2)$$

Consider handover k from source S_i to successor S_j , triggered at t_k . The remaining source window provides the migration budget $B_k = T_{host}(S_i, t_k)$. A migration mechanism produces a completion time $t_{mig,k}$ and an execution interruption d_k . To complete the handover safely, it must satisfy $t_{mig,k} \leq B_k$, and the selected successor must be SLO-compliant when authority transfers.

4.1.3 Steady-State Feasibility. Because migration recurs throughout $\mathcal{T}$, its cost must be evaluated cumulatively rather than as an isolated event. For n handovers, the fraction of operational time occupied by migration is

$$\phi_{mig} = \frac{\sum_{k=1}^n t_{mig,k}}{|\mathcal{T}|} \approx \frac{\bar{t}_{mig}}{\bar{T}_{host}}, \quad (3)$$

where the approximation applies when each hosting window induces one handover. Migration may overlap application execution, so ϕ_{mig} captures sustained link and compute occupancy rather than service downtime. If only execution pauses are considered, they yield the diagnostic $A_d = 1 - \sum_k d_k / |\mathcal{T}|$; A_d equals the availability in Equation 1 only when other latency violations are absent.

4.1.4 Contract. Together, the continuity invariants and operational constraints define the mechanism-independent VSR contract. Each migration must complete within its budget and commit to an SLO-compliant successor. Across the execution schedule, availability must remain above A_{min} and migration occupancy below the application-defined limit ϵ . Accordingly, feasibility requires

$$\forall k : \begin{cases} t_{mig,k} \leq B_k, \\ \zeta_j(t_k + t_{mig,k}) \leq S_{max}, \\ A \geq A_{min}, \\ \phi_{mig} \leq \epsilon. \end{cases} \quad (4)$$

4.2 Virtual Stationarity Runtime Framework

The model specifies what a handover must achieve, but not which component is responsible for each obligation. As shown in Figure 3, the framework relates an external orbital orchestrator, a compatible Execution Engine, and the Migration Membrane. It deliberately separates the external decision of *when and where* to migrate from the runtime mechanisms that determine *how* execution continues.

4.2.1 Components. Our framework distinguishes three core components to enable recurrent migrations.

External Orbital Orchestrator. The orchestrator predicts host $\mathcal{H}(t)$ and network evolution, applies Equation 4 and Equation 7, and supplies successor S_j , trigger t_k , and migration budget B_k . It thereby defines the admissible handover, while constellation prediction, host-selection policy, and routing remain outside the VSR. The runtime contributes to Equation 4 by reducing migration time $t_{mig,k}$ and interruption d_k within the supplied budget.

Execution Engine. The framework does not require a particular bytecode or managed runtime, but the engine must provide two

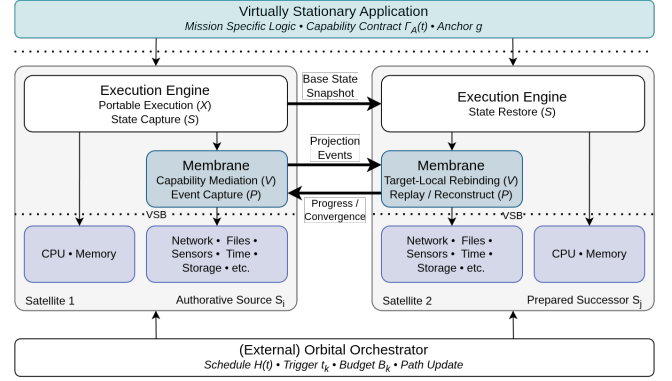


Figure 3: Virtual stationarity runtime. External orchestration supplies the selected host and trigger. The Execution Engine supplies portable execution and state capture; the Migration Membrane preserves capabilities, projects continued execution, and controls authority transfer.

properties. *Portable execution* (X) represents code and control state independently of the source OS and ISA. *State capture and restoration* (S) exposes safe points and a complete, relocatable representation of the state required to continue X . The engine also exposes the application-memory mutations caused by an intercepted system call. These are integration requirements.

Migration Membrane. The membrane binds to the engine at the Virtual Stationarity Boundary (VSB), separating logical execution from native resources and effects. It contributes Capability Virtualization (V) and Continuous State Projection (P), coordinates the handover protocol, and enforces authority exclusivity. Each realization operates in one of four modes: **LIVE** performs normal authoritative execution; **RECORD** performs authoritative execution while emitting continuation events; **REPLAY** reconstructs those events without external effects; and **INACTIVE** holds prepared bindings that cannot be invoked.

4.2.2 Handover Protocol. Our framework structures recurrent migration into four distinct phases.

Prepare. The membrane validates the supplied migration parameters and requests a source safe point. With no system call in flight, S captures the portable base S_{base} . The source initializes its opcode counter relative to this cut; the target initializes the corresponding counter after restoring the same base.

Reconstruct. The base is transferred while the source continues in **RECORD**. The target restores S_{base} , starts a receiver for continuation events, and uses V to establish target-local capability bindings. These bindings remain **INACTIVE** and thus cannot produce effects.

Project. After restoration, the target enters **REPLAY**. The source continues intercepting system calls and emitting their results and memory mutations; the target consumes the events and reconstructs the same transitions without invoking its local bindings. Projection continues until replay lag satisfies the commit threshold.

Commit. The source quiesces at an opcode boundary, closes its effect gate, transitions from **RECORD** to **INACTIVE**, and emits a terminal **END** event containing its execution fence. After consuming all preceding events and reaching the same fence, the target activates

its bindings and transitions from Replay to Live. The runtime then requests the orchestrator's path update.

4.3 Migration Membrane Mechanisms

A portable snapshot can reconstruct a stopped computation, but it neither preserves the meaning of host-bound resources nor captures state accumulated while the snapshot is in transit. The Migration Membrane $\mathcal{M} = \langle \mathcal{V}, \mathcal{P} \rangle$ addresses both gaps, where $\mathcal{V}$ mediates the application's relationship with its physical environment and $\mathcal{P}$ maintains a non-authoritative projection of its evolving execution.

4.3.1 Capability Virtualization ($\mathcal{V}$). Application A expresses each host-dependent operation as a logical capability $c \in C_A$ with a stable name independent of any native handle or implementation. Each satellite S_i has a static inventory C_i of capabilities it can implement. The runtime uses the same logical namespace on every host and changes only the local binding $\beta_i(c)$.

Declaration. The application capability contract is

$$\Gamma_A(t) = \langle C_A^{req}(t), C_A^{opt}(t) \rangle, \quad C_A^{req}(t) \cup C_A^{opt}(t) = C_A, \quad (5)$$

The initial capability set can be declared explicitly or extracted by inspecting the application's source-level declarations, intermediate representation, or compiled external-call interface. Capabilities are required by default. During execution, the application may call $\text{SetPolicy}(c, p)$, with $p \in \{\text{required}, \text{optional}\}$, to update the contract stored in logical state C . Policy changes are deterministic control operations applied by both realizations and require no distinct continuation-event type. For optional capabilities, the membrane exposes a function so the application can adapt its behavior according to the active host.

$$\text{Available}(c) = \begin{cases} \text{true}, & c \in C_{\mathcal{H}(t)}, \\ \text{false}, & \text{otherwise}, \end{cases} \quad (6)$$

Admission. The orbital orchestrator admits only candidates satisfying

$$\text{Compatible}(A, S_i, t) \iff C_A^{req}(t) \subseteq C_i. \quad (7)$$

Required capabilities therefore constrain placement in addition to Equation 4. Optional capabilities do not constrain placement; they constrain application behavior through Equation 6. The contract is sampled when the successor is selected, and later policy changes affect the next placement decision.

Preparation. During reconstruction, the target evaluates $\beta_j(c) \leftarrow \text{Rebind}_j(c)$ for every capability. Available capabilities obtain host-local resources; a missing optional capability maps to the defined UNAVAILABLE behavior, whereas a missing required capability invalidates admission. Preparation establishes native resources but leaves all target bindings INACTIVE; it grants neither execution authority nor permission to produce external effects.

Activation. While the source remains authoritative, system calls invoke $\beta_i(c)$ in LIVE or RECORD. The target in REPLAY returns recorded outcomes without invoking $\beta_j(c)$. Only after the handover protocol commits are the source bindings revoked and the target bindings activated, so subsequent calls use target-local implementations. Consequently, logical capability identity persists across hosts while only the authoritative realization may produce new external effects.

Algorithm 1 VSB-mediated capability invocation

Require: Capability c , operation f , arguments a , mode $m \in \{\text{Live}, \text{Record}, \text{Replay}, \text{Inactive}\}$, binding β_i , target FIFO Q

```

1: if  $m = \text{Replay}$  then
2:    $e \leftarrow Q.\text{DequeueCallBlocking}()$ 
3:   if  $e = \text{finalized} \vee e.s \neq \text{Opld}(c, f)$  then
4:     return ABORT
5:    $\text{Apply}(e.\Delta M); h_t \leftarrow h_t + 1$ 
6:   return  $e.r$  without invoking  $\beta_i(c)$ 
7: if  $m = \text{Inactive}$  then
8:   return ABORT
9: if  $\beta_i(c) = \perp$  then
10:   $r \leftarrow \text{Unavailable}; \Delta M \leftarrow \emptyset$ 
11: else if  $m = \text{Record}$  then
12:   $(r, \Delta M) \leftarrow \text{InvokeAndCapture}(\beta_i(c), f, a)$ 
13: else
14:   $r \leftarrow \text{Invoke}(\beta_i(c), f, a)$ 
15: if  $m = \text{Record}$  then
16:   $h_s \leftarrow h_s + 1$ 
17:   $\text{Emit}(\langle h_s, \text{Opld}(c, f), r, \Delta M \rangle)$ 
18: return  $r$ 

```

4.3.2 Continuous State Projection ($\mathcal{P}$). Continuous State Projection reconstructs a target from a portable base and an ordered continuation while the source remains authoritative. We assume reliable FIFO channels and single-threaded modules whose host interactions occur synchronously through the VSB. Execution between VSB crossings is deterministic.

State capture. At trigger t_0 , the source reaches a safe point with no system call in flight. $\mathcal{S}$ captures $S_{base} = \text{Capture}(\sigma(t_0))$, while paused, and the source initializes opcode counter $q_s = 0$. The source realization then enters RECORD before resuming. The target initializes $q_t = 0$ after restoring the same cut. The execution-engine contract includes mutation tracking for intercepted system calls. Before invoking a native operation, the tracker is reset. Runtime paths that permit the operation to modify application memory register each affected pointer and length. Immediately after the call returns, a collect-and-reset operation converts those pointers to relocatable memory offsets and copies the resulting bytes:

$$\Delta M_k = \{ \langle o_{k,\ell}, n_{k,\ell}, b_{k,\ell} \rangle \}_{\ell=1}^{m_k}, \quad (8)$$

where $o_{k,\ell}$ is an offset, $n_{k,\ell}$ the region length, and $b_{k,\ell}$ the bytes visible after the call. An implementation may expose this as an engine API or as an internal additional return value of native invocation. The continuation event is

$$e_k = \langle k, s_k, r_k, \Delta M_k \rangle, \quad s_k = \text{Opld}(c_k, f_k), \quad (9)$$

where k is the FIFO event counter, s_k the operation signature, and r_k is the direct return value. Separating r_k from ΔM_k is essential for reads: a file or socket read may return only a status or byte count in r_k , while the received content resides in the memory regions of ΔM_k . The source enqueues e_k before returning to the application, and an asynchronous sender preserves event order.

Transfer and replay. Base state and continuation events use independent channels. A receiver places events into the ordered buffer

Q , so replay never reads from the transport directly. Algorithm 1 applies each event without invoking the target binding; mismatches or buffer overflow abort reconstruction while the source remains authoritative. The terminal END event is handled separately as the authority fence.

Convergence. Let h_s be the number of emitted events and h_t the number acknowledged after replay. Their difference is the observable lag $L = h_s - h_t$. Then $T_0 = t_{ovh} + \Phi/C$ denotes the time to capture, transfer, and restore a base snapshot of size Φ at effective bandwidth C . Events accumulate at rate r_{sys} , giving an initial replay backlog $Q_0 = r_{sys}T_0$. While the source continues, a target processing events at rate μ drains this backlog at rate $\mu - r_{sys}$. Hence

$$t_{mig}^M = T_0 + \frac{Q_0}{\mu - r_{sys}} = \left(t_{ovh} + \frac{\Phi}{C}\right) \frac{\mu}{\mu - r_{sys}}, \quad r_{sys} < \mu. \quad (10)$$

Convergence requires $r_{sys} < \mu$. For $r_{sys} \ll \mu$, migration time is approximately linear in Φ at slope $1/C$ and independent of memory-dirty rate. As r_{sys} approaches μ , the continuation backlog ceases to converge. Pre-copy instead approaches $t_{pre} \approx (\Phi/C)/(1 - r_{dirty}/C)$ and diverges as r_{dirty} approaches C [9]. If $r_{sys} \geq \mu$ or predicted $t_{mig,k}^M > B_k$, the framework aborts target reconstruction and leaves the source authoritative.

Authority transfer. The condition $L \leq L_{conv}$ only triggers commit; it does not prove that source and target are at the same instruction state. Both realizations therefore maintain the opcode counters initialized at the base cut and increment them after every completed opcode. At commit, the source quiesces at an opcode boundary and fixes fence $\langle q^*, i^* \rangle$, where q^* is its completed-opcode count and i^* identifies the opcode at that boundary. It then changes from RECORD to INACTIVE, closing the source effect gate before any target activation. The sender flushes all ordinary events and appends the final packet

$$e_{end} = \langle \text{END}, q^*, i^* \rangle. \quad (11)$$

FIFO order makes END an explicit statement that no further source-produced system-call outcome can follow. The target receiver stores the fence separately and marks Q finalized after all preceding events have been inserted. The target remains in REPLAY, applies the remaining events, and checks $\langle q_t, i_t \rangle$ after each opcode. It transitions from REPLAY to LIVE and activates β_j only when Q is empty and $\langle q_t, i_t \rangle = \langle q^*, i^* \rangle$. If the target passes the counter or the opcode identifier differs, commit fails without enabling effects. Thus, source revocation precedes target activation and there is never more than one effect-producing realization. Stream closure remains a transport indication; Equation 11 identifies the exact execution cut. After activation, the runtime requests the orchestrator's path update. Failures before source revocation abort the projection; recovery after revocation is outside scope.

4.4 Prototype Implementation

We integrate our approach with the WebAssembly Micro Runtime (WAMR). An interpreter hook after each opcode checks whether migration has been triggered. At the selected boundary, the interpreter thread captures application state together with stack and shadow-stack frames. To minimize this pause, it creates a read-only

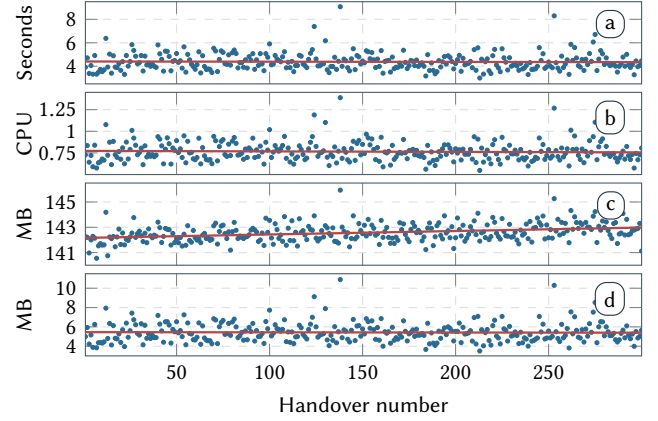


Figure 4: Stability across 300 accelerated handovers with a 30 s trigger interval. Points report (a) per-handover migration time, (b) aggregate CPU consumption across both hosts, (c) post-cleanup memory, and (d) transferred continuation events. Lines show least-squares trends.

MAP_PRIVATE view of the linear-memory memfd backing object [20]. Copy-on-write preserves the base while the source resumes and the image is transmitted. The target restores the relocatable structures, providing X and S across operating systems and ISAs.

We adapt WAMR's native-import boundary to realize $\mathcal{V}$. At module load, the runtime derives logical capabilities from imported module/function pairs. Missing optional WASI functions return `__WASI_ERRNO_ENOSYS`. The evaluated UDP binding captures its descriptor slot and port, creates a target socket, and updates WAMR's descriptor table to the new native socket.

For $\mathcal{P}$, the mutation tracker is reset before native dispatch and records the linear-memory regions written by the system call. After dispatch, the runtime serializes each record as offset, length, and payload alongside a function-signature hash and two return cells. The execution thread appends these events to ping-pong buffers; a flusher thread swaps them at most every 500 μ s and transmits completed batches without blocking execution on network I/O. On the target, a receiver thread inserts events into a bounded FIFO buffer consumed by Algorithm 1, while periodic acknowledgements expose h_t to the source. At commit, the source stops at the next opcode, flushes the final batch, and appends END with the source opcode counter and identifier. The target opcode hook activates execution only at the matching local tuple. The prototype currently supports single-threaded modules; the evaluated calls modify one contiguous output region. Anonymized source code and build scripts for our adapted WAMR runtime prototype are available for review¹.

5 Evaluation

We evaluate our prototype on a physical testbed of four Raspberry Pi 5 Single-Board Computers (SBCs) connected by an unmanaged Gigabit switch. Each node has a quad-core 2.4 GHz ARMv8 processor, 8 GB of RAM, an active cooling unit, and runs Raspberry Pi OS (Debian 13). Raspberry Pi devices represent Commercial

¹<https://anonymous.4open.science/r/iot-2026-wamr-migration-membrane-8413/>

Off-The-Shelf (COTS) payload classes rather than specific flight hardware. Devices from these families have flown on satellites before [10, 14, 40]. The evaluation follows our three contributions: §5.1 and §5.2 test steady-state execution and semantic continuity, §5.3 quantifies the projection bounds in Equation 10, and §5.4 and §5.5 evaluate cross-ISA execution and orbital feasibility using t_{mig} , d , ϕ_{mig} , and A_d . Because path updates are external to VSR, A_d measures runtime interruption rather than the end-to-end availability A in Equation 1.

5.1 Steady-State Correctness and Stability

To evaluate whether repeated migration accumulates computational, memory, or continuation debt, we conducted an accelerated long-horizon experiment on the two-node testbed. We triggered a migration every 30 seconds for 300 consecutive handovers, resulting in approximately 2.5 hours of continuous operation. This cadence is deliberately more demanding than the expected orbital handover frequency. Nevertheless, every migration completed before the following trigger.

Table 1: Operational stability and longevity metrics across recurrent handovers, comparing overall execution with the initial and final handover windows.

Metric	Median	p_{95}	First 50	Last 50	Change
Mig. time (t_{mig}^M)	4,241 ms	5,535 ms	4,219 ms	4,181 ms	−38 ms
Downtime	5.52 ms	5.96 ms	3.76 ms	5.51 ms	+1.75 ms
CPU (core-s)	0.748	0.938	0.733	0.738	+0.005
Memory	142.5 MB	143.8 MB	142.2 MB	142.9 MB	+0.65 MB
Base transfer	135.5 MB	135.7 MB	135.5 MB	135.5 MB	−0.01 MB
Event transfer	5.29 MB	7.04 MB	5.22 MB	5.27 MB	+0.05 MB
Overhead (ϕ_{mig})	14.083 %	18.482 %	14.067 %	13.871 %	−0.196 %
Availability (A_d)	99.982 %	99.995 %	99.982 %	99.982 %	+0.000 %
Duplications	0	0	0	0	0

Figure 4 and Table 1 show no material drift in migration time, CPU consumption, memory, or continuation traffic. Although median downtime increased by 1.75 ms, it remained below 6 ms and did not increase with migration time. Migration time remained stable at a median of 4.24 s ($p_{95} = 5.54$ s), while CPU consumption remained at 0.75 core-seconds ($p_{95} = 0.94$). Post-cleanup memory had a median of 142.5 MB and increased by only 0.27 MB per 100 handovers, corresponding to less than 0.6 % over the complete run. Continuation traffic was similarly stable, with a median of 5.29 MB compared with 135.47 MB for the base snapshot, and exhibited no positive trend. Median and p_{95} downtime were 5.52 ms and 5.96 ms, respectively; aggregating all interruptions over the experiment yields $A_d = 99.9857\%$. These results indicate that recurrent handovers neither increase migration cost nor accumulate unresolved continuation state, supporting the claim that virtual stationarity can sustain steady-state operation over repeated migrations.

5.2 Semantic Continuity

We evaluate state, effect, and capability continuity across 100 recurrent handovers. The workload incremented a sequence number, obtained a nondeterministic value through `random_get`, incorporated both into a rolling 64-bit hash, and emitted the resulting

tuple through `fd_write`. During Replay, the target reconstructed recorded returns and memory mutations without invoking the output binding; after commit, execution continued through the target-local binding. An offline verifier combined the outputs from both hosts and checked the complete sequence and hash chain.

Across 100 handovers and 176,801 replayed events, the verifier found no sequence gaps, duplicate outputs, or hash-chain mismatches. The gap-free sequence demonstrates state continuity, while the absence of duplicates provides evidence of effect continuity and authority exclusivity for the evaluated output path. Because every hash depends on a nondeterministic `random_get` result, the intact chain shows that $\mathcal{P}$ reproduced both system-call outcomes and their memory mutations. Continued output through the target-local `fd_write` binding demonstrates capability continuity under $\mathcal{V}$ for the evaluated workload.

5.3 Scaling and Convergence Bounds

Equation 10 predicts that Continuous State Projection scales with base-state volume at low event rates and converges only while $r_{sys} < \mu$. We evaluate both properties against checkpoint/restore and pre-copy on two physical nodes under realistic ISL profiles, varying base-state volume, memory-write rate, and intercepted host-call rate. Each intercepted call produces one continuation event, so its rate corresponds to r_{sys} . Post-copy is omitted because its predecessor-dependent completion times exceed the recurrent handover budgets considered here.

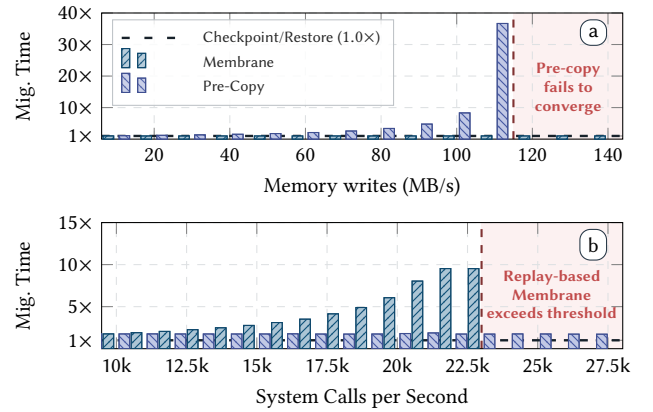


Figure 5: Continuous State Projection bounds. (a) Pre-copy fails as memory writes approach transfer capacity, whereas membrane migration remains invariant. (b) Replay-based membrane migration grows as event production approaches replay capacity and eventually exceeds the reference budget.

Figure 5a shows that membrane migration varies by less than 1% across write loads of 10–140 MB/s, whereas pre-copy ceases to converge as the dirtying rate approaches available bandwidth. In a separate 200–1,000 MB base-state sweep, membrane and checkpoint/restore yield linear slopes of 9.06 and 8.96 ms/MB, respectively ($R^2 > 0.999$), corresponding to an effective bandwidth of $C \approx 110$ MB/s. Across this sweep, membrane downtime remains within 2–5 ms. On the other hand, Figure 5b confirms the event-rate bound predicted

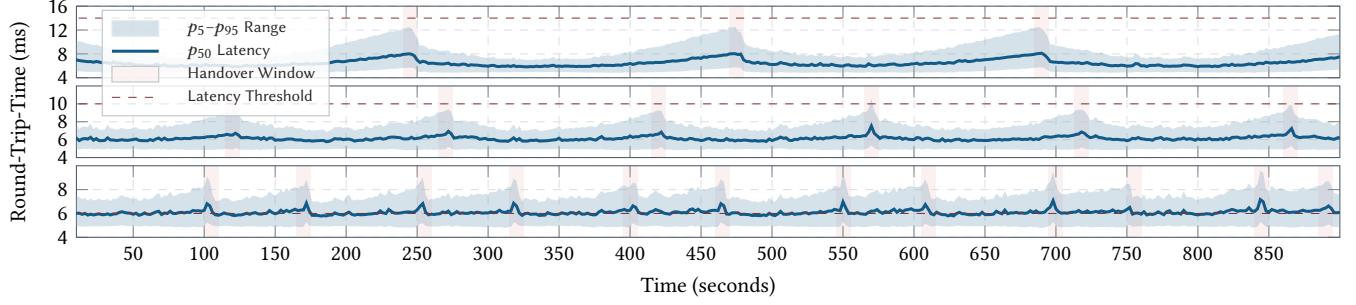


Figure 6: End-to-end latency during continuous migration for 14 ms, 10 ms, and 6 ms target SLOs on a 1,600-satellite constellation. Blue lines show median RTT; tighter targets shorten the median interval between handovers to 242 s, 151 s, and 83 s, respectively.

by Equation 10. Fixing Φ/C from the base-state experiment and fitting the remaining parameters yields $\mu = 24.1 \times 10^3$ events/s and $t_{ovh} = 136$ ms. Measured durations deviate from the model by 1.3% on average over 1,000–21,000 calls/s, while a fit trained on 1k–15k calls/s predicts the withheld 16k–21k measurements within 0.8% mean error. Using the 15 s beam-switch period reported by Zhao et al. [42] as a demanding reference rather than a universal budget B_k , the model predicts a breach at 20.2k calls/s. Here, μ denotes the end-to-end catch-up rate, including event ingestion, application of ΔM , and deterministic execution between events.

5.4 Portability

The framework requires its Execution Engine to provide portable execution ($\mathcal{X}$). We validate this requirement through bidirectional migration between an x86-64 host (Lenovo ThinkPad, 4.31 GHz, 32 GB RAM) and an ARM64 node (Raspberry Pi 5). In all $N = 100$ trials, the workload resumed on the target without application changes. The result confirms that the selected WAMR engine satisfies $\mathcal{X}$ for the evaluated architecture pair; it does not assume that arbitrary native runtimes are architecture-independent.

5.5 Orbital Feasibility

We evaluate repeated handovers over multi-minute runs using a constellation simulator based on the Simplified General Perturbations 4 (SGP4) model [38]. The simulator calculates satellite trajectories and supplies the migration schedule and corresponding ISL/GSL profiles to the physical testbed. Two Raspberry Pi 5 nodes alternate as source and successor. This experiment evaluates the framework’s runtime lifecycle under an externally supplied schedule, rather than the optimality of successor selection.

Table 2: Orbital runtime metrics (medians); A_d includes run-time interruption only.

Latency	Mig. Time	Downtime	On Host	Overh. (ϕ_{mig})	Avail. (A_d)
14 ms	3212 ms	4.09 ms	242 s	1.339 %	99.998 %
10 ms	3229 ms	3.85 ms	151 s	2.267 %	99.997 %
6 ms	3402 ms	2.10 ms	83 s	4.403 %	99.997 %

We evaluate target latency profiles of 14, 10, and 6 ms, with the latter approaching the physical lower bound of the modeled links.

Figure 6 shows an active XR workload receiving approximately 4,000 packets/s. Across the 900 s traces, median RTT exhibits no upward drift over successive handovers. Migrations complete in 2.8–4.5 s within observed handover intervals of 83–242 s, leaving substantial budget under the condition $t_{mig,k} \leq B_k$. The similar migration times across profiles show that tighter latency targets increase handover frequency rather than individual migration cost.

The same logical UDP descriptor is rebound and remains usable across every handover, demonstrating capability continuity. Across 20 aligned handovers, the time-averaged p_{50} packet-loss trace is 2.01% during the handover window, compared with 1.97% before and 1.99% after. Its maximum within-window p_{50} is 2.22%, while p_{95} peaks at 5.10%. Median interruptions of 2.10–4.09 ms yield $A_d \geq 99.997\%$, while ϕ_{mig} ranges from 1.339% to 4.403% (Table 2). Because A_d excludes external path updates and ordinary latency violations, it does not establish end-to-end availability A .

6 Conclusion & Future Work

We presented VSR, a runtime framework that treats recurrent orbital migration as a steady-state execution primitive. Starting from a mechanism-independent feasibility model, the framework separates external host selection and scheduling from runtime handover. Its Migration Membrane combines Capability Virtualization to rebind host-dependent resources with Continuous State Projection to reconstruct an evolving execution on the successor. An explicit authority fence completes the handover without retaining predecessor dependence or permitting both hosts to produce external effects. Our prototype demonstrated that this model can operate across heterogeneous hosts. Across 300 recurrent handovers, p_{95} downtime remained below 6 ms without cumulative growth in migration time or continuation state. The prototype also preserved the evaluated state, capability, and effect continuity properties and resumed successfully across x86-64 and ARM64. Under simulated orbital schedules, migrations completed within the available handover intervals while maintaining bounded migration occupancy and downtime-only availability of $A_d \geq 99.997\%$.

Future work will incorporate orbital, network, energy, and thermal constraints into external host selection. We also plan to investigate continuously active state projection for constellations requiring higher handover frequencies and broader classes of virtualized capabilities.

References

- [1] Sima Attar-Khorasani, Lincoln Sherpa, Matthias Lieber, and Siavash Ghiasvand. 2025. Seamless Transitions: A Comprehensive Review of Live Migration Technologies. <https://arxiv.org/abs/2512.10979v1>
- [2] Tom Bergan, Nicholas Hunt, Luis Ceze, and Steven D. Gribble. 2010. Deterministic process groups in dOS. In *Proceedings of the 9th USENIX conference on Operating systems design and implementation (OSDI'10)*. USENIX Association, USA, 177–191.
- [3] Debopam Bhattacharjee, Simon Kassing, Melissa Licciardello, and Ankit Singla. 2020. In-orbit Computing: An Outlandish thought Experiment?. In *Proceedings of the 19th ACM Workshop on Hot Topics in Networks (HotNets '20)*. Association for Computing Machinery, New York, NY, USA, 197–204. doi:10.1145/3422604.3425937
- [4] Debopam Bhattacharjee and Ankit Singla. 2019. Network topology design at 27,000 km/hour. In *Proceedings of the 15th International Conference on Emerging Networking Experiments And Technologies*. ACM, Orlando Florida, 341–354. doi:10.1145/3359989.3365407
- [5] Nilotpal Biswas, Anamitra Mukherjee, and Samit Bhattacharya. 2024. “Are you feeling sick?” – A systematic literature review of cybersickness in virtual reality. *Comput. Surveys* 56, 11 (June 2024), 284:1–284:38. doi:10.1145/3670008
- [6] M. Bozyigit and M. Wasiq. 2001. User-level process checkpoint and restore for migration. *SIGOPS Oper. Syst. Rev.* 35, 2 (April 2001), 86–96. doi:10.1145/377069.377091
- [7] Vincent Charpentier, Nina Slamnik-Kriještorac, Giada Landi, Matthias Caenepel, Olivier Vasseur, and Johann M. Marquez-Barja. 2024. Paving the way towards safer and more efficient maritime industry with 5G and Beyond edge computing systems. *Computer Networks* 250 (Aug. 2024), 110499. doi:10.1016/j.comnet.2024.110499
- [8] Yang Chen. 2015. Checkpoint and Restore of Micro-service in Docker Containers. doi:10.2991/icmii-15.2015.160
- [9] Christopher Clark, Keir Fraser, Steven Hand, Jacob Gorm Hansen, Eric Jul, Christian Limpach, Ian Pratt, and Andrew Warfield. 2005. Live Migration of Virtual Machines. In *2nd Symposium on Networked Systems Design and Implementation (NSDI '05)*. USENIX Association, Boston, MA, 273–286. <https://www.usenix.org/conference/nsdi-05/live-migration-virtual-machines>
- [10] Angela Cratere, Leandro Gagliardi, Gabriel A. Sanca, Federico Golmar, and Francesco Dell'Olio. 2024. On-Board Computer for CubeSats: State-of-the-Art and Future Trends. *IEEE Access* 12 (2024), 99537–99569. doi:10.1109/ACCESS.2024.3428388
- [11] CRIU Project. 2021. CRIU and the Linux Kernel. CRIU documentation, revision 5155. Last edited 20 April 2021. https://criu.org/index.php?title=Linux_kernel&oldid=5155
- [12] Bradley Denby and Brandon Lucia. 2020. Orbital Edge Computing: Nanosatellite Constellations as a New Class of Computer System. In *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems*. ACM, Lausanne Switzerland, 939–954. doi:10.1145/3373376.3378473
- [13] George W. Dunlap, Samuel T. King, Sukru Cinar, Murtaza A. Basrai, and Peter M. Chen. 2002. ReVirt: Enabling Intrusion Analysis through Virtual-Machine Logging and Replay. In *Proceedings of the 5th Symposium on Operating Systems Design and Implementation (OSDI '02)*. USENIX Association, 211–224. <https://www.usenix.org/conference/osdi-02/revirt-enabling-intrusion-analysis-through-virtual-machine-logging-and-replay>
- [14] Alireza Furutanpey, Qiyang Zhang, Philipp Raith, Tobias Pfandzelter, Shangguang Wang, and Schahram Dustdar. 2025. FOOL: Addressing the Downlink Bottleneck in Satellite Computing With Neural Feature Compression. *IEEE Transactions on Mobile Computing* 24, 8 (Aug. 2025), 6747–6764. doi:10.1109/TMC.2025.3544516
- [15] Keerthana Govindaraj and Alexander Artemenko. 2018. Container Live Migration for Latency Critical Industrial Applications on Edge Computing. In *2018 IEEE 23rd International Conference on Emerging Technologies and Factory Automation (ETFA)*, Vol. 1. 83–90. ISSN: 1946-0759. doi:10.1109/ETFA.2018.8502659
- [16] Tianzhang He and Rajkumar Buyya. 2023. A Taxonomy of Live Migration Management in Cloud Computing. *Comput. Surveys* 56, 3 (2023), 56:1–56:33. doi:10.1145/3615353
- [17] Lawrence J. Hettinger, Kevin S. Berbaum, Robert S. Kennedy, William P. Dunlap, and Margaret D. Nolan. 1990. Vection and Simulator Sickness. *Military Psychology* 2, 3 (Sept. 1990), 171–181. doi:10.1207/s15327876mp0203_4
- [18] Michael R. Hines, Umesh Deshpande, and Kartik Gopalan. 2009. Post-copy live migration of virtual machines. *ACM SIGOPS Operating Systems Review* 43, 3 (July 2009), 14–26. doi:10.1145/1618525.1618528
- [19] Kiranpreet Kaur, Fabrice Guillemin, and Francoise Sailhan. 2022. Container placement and migration strategies for cloud, fog, and edge data centers: A survey. *International Journal of Network Management* 32, 6 (2022), e2212. doi:10.1002/nem.2212
- [20] Linux man-pages project. 2026. memfd_create(2) - Linux Manual Page. Linux man-pages 6.18. Manual-page revision dated 8 February 2026. https://man7.org/linux/man-pages/man2/memfd_create.2.html
- [21] Haikun Liu, Hai Jin, Xiaofei Liao, Liting Hu, and Chen Yu. 2009. Live migration of virtual machine based on full system trace and replay. In *Proceedings of the 18th ACM international symposium on High performance distributed computing (HPDC '09)*. Association for Computing Machinery, New York, NY, USA, 101–110. doi:10.1145/1551609.1551630
- [22] Cynthia Marcelino, Leonard Guelmino, Thomas Pusztai, and Stefan Nastic. 2025. Databelt: A Continuous Data Path for Serverless Workflows in the 3D Compute Continuum. *Journal of Systems Architecture* 168 (Nov. 2025), 103577. doi:10.1016/j.sysarc.2025.103577
- [23] Jonathan McDowell. 2026. Jonathan's Space Report | Space Statistics. <https://planet4589.org/space/stats/active.html>
- [24] Zili Meng, Tingfeng Wang, Yixin Shen, Bo Wang, Mingwei Xu, Rui Han, Honghao Liu, Venkat Arun, Hongxin Hu, and Xue Wei. 2023. Enabling High Quality Real-Time Communications with Adaptive Frame-Rate. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI '23)*. USENIX Association, Boston, MA, 1429–1450. <https://www.usenix.org/conference/nsdi23/presentation/meng>
- [25] Costas Michaelides, Miguel Casasnovas, Daniele Marchitelli, and Boris Bellalta. 2023. VR Gaming Dataset - Multi-user Tests. Dataset, Zenodo. Published 20 July 2023. doi:10.5281/zenodo.8169785
- [26] Shripad Nadgowda, Sahil Suneja, Nilton Bila, and Canturk Isci. 2017. Voyager: Complete Container State Migration. In *2017 IEEE 37th International Conference on Distributed Computing Systems (ICDCS)*. 2137–2142. ISSN: 1063-6927. doi:10.1109/ICDCS.2017.91
- [27] National Aeronautics and Space Administration. 2023. SCA Glossary. NASA Space Communications and Navigation Program. Last updated 22 September 2023. <https://www.nasa.gov/reference/scan-glossary/>
- [28] Stephen Palmisano, Robert S. Allison, Joel Teixeira, and Juno Kim. 2023. Differences in virtual and physical head orientation predict sickness during active head-mounted display-based virtual reality. *Virtual Reality* 27, 2 (June 2023), 1293–1313. doi:10.1007/s10055-022-00732-5
- [29] Tobias Pfandzelter and David Bernbach. 2024. Komet: A Serverless Platform for Low-Earth Orbit Edge Services. In *Proceedings of the 2024 ACM Symposium on Cloud Computing (SoCC '24)*. Association for Computing Machinery, New York, NY, USA, 866–882. doi:10.1145/3698038.3698517
- [30] Carlo Puliafito, Carlo Vallati, Enzo Mingozzi, Giovanni Merlino, Francesco Longo, and Antonio Puliafito. 2019. Container Migration in the Fog: A Performance Evaluation. *Sensors (Basel, Switzerland)* 19, 7 (March 2019), 1488. doi:10.3390/s19071488
- [31] Chenghao Rong, Jessie Hui Wang, Jilong Wang, Yipeng Zhou, and Jun Zhang. 2024. Live Migration of Video Analytics Applications in Edge Computing. *IEEE Transactions on Mobile Computing* 23, 3 (March 2024), 2078–2092. doi:10.1109/TMC.2023.3246539
- [32] Reinhard Scheuer, Yibo Pi, and Xudong Wang. 2026. ReSync: Coordinated Live-Migration for Stateful Containers in Mobile Edge Computing. *IEEE Transactions on Mobile Computing* (2026), 1–17. doi:10.1109/TMC.2026.3716750
- [33] Megha Sharma, Abhinav Tomar, and Abhishek Hazra. 2024. Edge Computing for Industry 5.0: Fundamental, Applications, and Research Challenges. *IEEE Internet of Things Journal* 11, 11 (June 2024), 19070–19093. doi:10.1109/JIOT.2024.3359297
- [34] Wissem Soussi, Gürkan Gür, and Burkhard Stiller. 2024. Democratizing Container Live Migration for Enhanced Future Networks - A Survey. *Comput. Surveys* 57, 4 (Dec. 2024), 97:1–97:37. doi:10.1145/3704436
- [35] SpaceX. 2024. Improving Starlink's Latency. Starlink technical update. Published 7 March 2024. <https://starlink.com/public-files/StarlinkLatency.pdf>
- [36] Jan-Philipp Stauffert, Florian Niebling, and Marc Erich Latoschik. 2018. Effects of Latency Jitter on Simulator Sickness in a Search Task. In *2018 IEEE Conference on Virtual Reality and 3D User Interfaces (VR)*. 121–127. doi:10.1109/VR.2018.8446195
- [37] Marvin M. Theimer, Keith A. Lantz, and David R. Cheriton. 1985. Preemptable remote execution facilities for the V-system. *SIGOPS Oper. Syst. Rev.* 19, 5 (Dec. 1985), 2–12. doi:10.1145/323627.323629
- [38] David Vallado and Paul Crawford. 2008. SGP4 Orbit Determination. doi:10.2514/6.2008-6770
- [39] Haonan Wu, Xiumei Yang, and Zhiyong Bu. 2024. Task Offloading With Service Migration for Satellite Edge Computing: A Deep Reinforcement Learning Approach. *IEEE Access* 12 (2024), 25844–25856. doi:10.1109/ACCESS.2024.3367128
- [40] Ruolin Xing, Mengwei Xu, Ao Zhou, Qing Li, Yiran Zhang, Feng Qian, and Shangguang Wang. 2024. Deciphering the Enigma of Satellite Computing with COTS Devices: Measurement and Analysis. In *Proceedings of the 30th Annual International Conference on Mobile Computing and Networking (ACM MobiCom '24)*. Association for Computing Machinery, New York, NY, USA, 420–435. doi:10.1145/3636534.3649371
- [41] Chenying Yu and Fei Huan. 2015. Live Migration of Docker Containers through Logging and Replay. Atlantis Press, 623–626. doi:10.2991/icmii-15.2015.106
- [42] Jinwei Zhao, Jack Baude, Ali Ahangarpour, Vaibhava Krishna Devulapalli, Sree Ganesh Lalitaditya Divakarla, Zhi-Li Zhang, and Jianping Pan. 2026. Demystifying Starlink Network Performance under Vehicular Mobility with Dynamic Beam Switching. Version 1, submitted 20 January 2026. arXiv:2601.13790 [cs.NI] <https://arxiv.org/abs/2601.13790v1>